

# Uporaba spodbujevanega učenja s povratno arhitekturo GRU za kompleksne gibe dvokolesnega mobilnega robota

Jaka Verk

University of Maribor, Faculty of Electrical Engineering and Computer Science  
Koroška cesta 46, 2000 Maribor, Slovenia  
E-pošta: jaka.verk@student.um.si

## Reinforcement Learning with a Recurrent Architecture GRU for Complex Movements of a Two-Wheeled Mobile Robot

*Two-wheeled robots capable of balancing on a single wheel present a substantially harder control problem than conventional two-wheel balancing, with no closed-form model for the tilted equilibrium. This paper presents, to the best of our knowledge, the first reinforcement learning based controller for sustaining a two-wheeled robot in a stable single-wheel stance, trained end-to-end with PPO using a compact Gated Recurrent Unit (GRU) architecture that implicitly encodes system dynamics through its hidden state. The network is sized for real-time deployment on an STM32F446RE microcontroller at 50 Hz; domain randomization and a three-stage roll-perturbation curriculum bridge the sim-to-real gap. The learned policy proves capable of achieving and maintaining balance across a wide range of initial roll angles, reaching near-100% survival probability within the trained  $\pm 11^\circ$  band around the  $47^\circ$  equilibrium stance.*

## 1 Uvod

Mobilni roboti, namenjeni za uporabo v nestrukturiranem naravnem okolju - gozdovi, gradbišča in področja naravnih nesreč, se pogosto poslužujejo uporabe koles v kombinaciji z kontroliranimi nogami, kjer žrtvujejo energijsko učinkovitost lažjemu prilagajanju terenu. Zadnje raziskave so prikazale, da se takšni kolesno-roboti lahko prilagajajo zahtevnemu okolju in imajo sposobnosti premagovanja ovir, vožnje po naklonini in zanesljivo delujejo v skrajnih okoljih, kot so gorske planote [1]. Krmilne strategije, ki poganjajo takšne robote so skoraj brez izjem učene s pomočjo spodbujevanega učenja (angl. RL, reinforcement learning), omogočeno s pomočjo GPU-pospešenim izvajanjem več tisoč instanc fizikalnih simulacij na enkrat. V [2] so pokazali, da lahko množično vzporedno učenje v kombinaciji z večfaznim učenjem naraščajoče težavnosti terena in motenj v nekaj minutah ustvari robustne politike hoje štirinožnih robotov ter da se politike, naučene izključno v simulaciji, brez dodatnega prilagajanja lahko neposredno prenesejo na fizično strojno opremo.

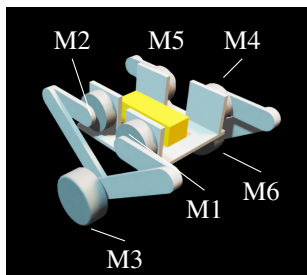
Nadaljnje raziskave so se posvetile ugotovitvam, katere arhitekture so najboljše za uporabo kot strategijo v RL [3]. Primerjali so usmerjene, povratne (LSTM, GRU),

transformerske in arhitekture s prostorom stanj (Mamba) na podlagi različnih nalog krmiljenja z RL ter ugotovili, da so pri nalogah, ki zahtevajo vzpostavljanje ravnotežja in stabilnosti pod delnim opazovanjem stanj, pri omejeni strojni opremi najboljša izbira povratne arhitekture kot sta GRU (angl. gated recurrent unit) in LSTM (angl. long short term memory).

V sorodnem delu [4] so zasnovali agilnega dvokolesnega robota z zaprto kinematično verigo petih členov, pri čemer nogi vsake strani poganjata dva motorja in kolesi neposredno gnana motorja. Stabilizacijo nagiba robota so dosegli s kaskadnim regulatorjem sestavljenim iz notranje zanke, ki uporablja kvadratični integralni regulator (LQI) in zunanje zanke, poslužujoče se PID. Pristop eksplicitno ne uporablja globokega učenja pri čemer robot v vseh manevrih ravnotežje ohranja na obeh kolesih hkrati.

V tem delu za dvokolesnega robota iz [4] uvedemo krmilno strategijo, ki vzdržuje trajno, statično pozo na enem samem kolesu, stanje bistveno zahtevnejše od dvokolesnega ravnotežja. S tem, kolikor nam je znano, predstavljamo prvo naučeno strategijo za ohranjanje ravnotežja dvokolesnega robota na enem kolesu.

Za reševanje obravnavane naloge ravnotežja smo predhodno preizkusili standardno usmerjeno nevronske mreže (angl. MLP, multilayer perceptron), vendar se je ta v prisotnosti senzornega šuma in časovnih zakasnitev izkazala za nestabilno ter ni konvergirala k zanesljivemu ravnotežju. Ti rezultati se skladajo z ugotovitvami študije [3], kjer avtorji ugotavljajo, da MLP mreže sicer omogočajo visoko učinkovitost, v okoljih z delno opazljivostjo pa odpovedo, zato smo kot osnovo strategije uporabili arhitekturo GRU z nevronske mreže, ki ji ni treba poznati niti eksplicitnega modela dinamike niti njegovih parametrov za posamezno konfiguracijo nog, s čimer nadaljujemo v smeri nadaljnega dela, ki ga predlagajo avtorji [4]. V nasprotju z GRU arhitekturami, primerjanimi v [3] zgolj v simulacijskih okoljih, je naša povratna politika načrtovana in naučena za dejansko namestitev na vgrajenem mikrokrmilniku (STM32) pri nadzorni frekvenci 50 Hz, kar zahteva izjemno majhno strategijo za namen prenosljivosti strategije iz simulacijskega okolja v resnično, računsko omejeno strojno opremo za razliko od [2], kjer so uporabili bistveno zmogljivejši vgrajeni računalnik.



Slika 1: USDC model robotske platforme z označenimi motorji M1–M6.

## 2 Robot in formulacija problema

Platforma iz [4] uporablja štiri motorje Xiaomi CyberGear (M1, M2, M4, M5) za gibljiv trup (nagib in višina), dva motorja DDSM115 (M3, M6) za pogon koles in stabilizacijo naklona, IMU BNO080 za orientacijo, baterijo ter mikrokontroler STM-32F446RE, na katerem teče regulacija (slika 1).

Motorji CyberGear so pozicijsko vodeni z impedančno regulacijo, kolesa DDSM115 pa tokovno oziroma navorno. Na podlagi dejanske platforme je bil ustvarjen poenostavljen 3D model v USDC (angl. USD Crate) formatu (slika 1). 3D model smo uvozili v IsaacLab [5], ki poleg geometrije opisuje tudi sklepe, pogone ter masne in vztrajnostne lastnosti povezanih teles.

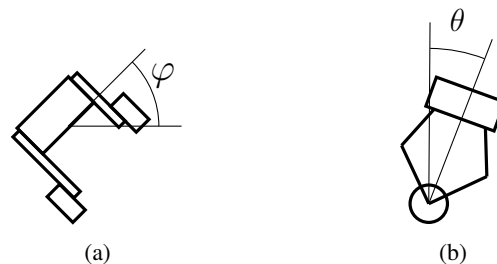
Glavni problem, ki ga je moral robot rešiti, je bilo ohranjanje bočno nagnjene nestabilne pozicije na enem kolesu z aktuatorji, krmiljenimi z nevronske mreže.

V našem problemu smo robotu določili dva glavna kota: nagib  $\varphi$  (angl. roll) (slika 2a) in naklon  $\theta$  (angl. pitch) (slika 2b). Nagib poteka okrog vzdolžne osi (vstran), medtem ko naklon poteka okrog prečne osi (naprej–nazaj).

Robot je bil v simulaciji postavljen na eno nogo, tako da je bilo le eno kolo v stiku s tlemi (M3), desna (zgornja) sklepa (M4, M5) sta v pokrčeni legi ( $0^\circ$ ), leva (spodnja) sklepa (M1, M2) pa v iztegnjeni legi  $45^\circ$ . Zložitev zgornjih in izteg spodnjih sklepov premakne težišče in omogoči stabilnost na enem kolesu, zato so zgornje noge čim bolj zložene, spodnje pa le toliko odmaknjene, da jih lahko še normalno krmilimo v obe smeri. V tej postavitvi mora regulator hkrati vzdrževati dve sklopljeni veličini. Naklon uravnava prek toka edinega talnega kolesa (M3), bočni nagib pa prek kotov vseh sklepov (M1, M2, M4, M5). Ker pa nima nobene avtoritete okoli navpične osi, je število neodvisnih krmilnih veličin manjše od števila prostostnih stopenj, zato je sistem podaktuiran. Ravnovesna lega določena z geometrijo oziroma lego težišča, je bila predvidena v nagibu  $47^\circ$ , a je nismo predpisali, temveč jo je moral regulator vzdrževati sam.

## 3 Učno okolje in zasnova regulatorja

Delno opazljiv markovski problem robota se izvaja v načrtno nepredvidljivem okolju v simulaciji in realnosti kar pomeni, da na robota vplivajo nepredvideni parametri kot so trenje, šum, zakasnitev senzorjev, itd. Za lažje naslavljanje tega izziva smo za arhitekturo nevronske mreže iz-



Slika 2: (a) Kot nagiba, (b) Kot naklona

brali nevronske mreže s povratno zanko (angl. RNN, recurrent neural network), z uporabo pomnilnih celic GRU. Celica GRU skozi skrito stanje kodira časovno dinamiko sistema, kar robotu omogoča sprotno kompenzacijo fizikalnih netočnosti brez ročno nastavljenih filtrov ali zgodovinskih podatkov. GRU tako deluje kot ucljiv, nelinearni filter, ki bistveno poveča robustnost zaprto-zančnega vodenja. Učenje nevronske mreže je potekalo z algoritmom proksimalne optimizacije strategije (angl. PPO, proximal policy optimization) v fizikalni simulaciji s korakom 1 ms, na 4000 vzporednih okoljih na enem GPU-ju, z adaptivno hitrostjo učenja in 64 koraki med posodobitvami uteži.

Agent prejme 6-razsežni vektor stanj (slika 3), jih procesira skozi prvi skriti sloj z 32 GRU celicami, nato pa čez 2 skrita sloja MLP z 32 nevroni in ReLU aktivacijsko funkcijo ter kot izhod vrne 5-razsežni vektor, ki je uporabljen kot vhod na treh motorjih robota. Prva komponenta izhodnega vektorja s pomočjo funkcije tanh določa tok talnega kolesa (DDSM115, M3), naslednji dve ciljni kot sprednjega in zadnjega levega sklepa (CyberGear M1 in M2) in zadnji dve ciljni kot sprednjega in zadnjega desnega sklepa (CyberGear M4 in M5). Dvignjeno kolo (M6) toka ne prejema, zato v akcijskem vektorju ne nastopi. Vsa opazovana stanja so normirana s fiksnimi, vnaprej določenimi vrednostmi, kar omogoča, da je aritmetika pri poznejši izvedbi na mikrokontrolerju identična simulacijski.

Nagrada za učenje nevronske mreže je zasnovana na podlagi nepoznanosti dejanskih kotov nagiba v ravnovesni legi, zato v nagradi ne predpišemo željenega kota nagiba, temveč kaznujemo kvadrat njegove kotne hitrosti, kar omogoči, da ravnovesno lego nevronska mreža odkrije sama. Ker smo naklon ravnovesne lege poznali vnaprej ( $0^\circ$ ), smo poleg kotne hitrosti kaznovali tudi po kotu samem. Nagrada ne vsebuje le opisanih členov, temveč tudi konstantno nagrado za preživetje, kazni za hitrost premika, velikost in gladkost sklepnihi ukazov, porabo toka, ob predčasni prekinitvi pa enkratno kazen  $-100$  (tabela 1, enačba 1).

Učenje strategije je potekalo 1300 iteracij, pri čemer se je razpon začetne perturbacije bočnega nagiba postopoma spreminjal skozi tri faze. Prva faza ( $\pm 5^\circ$ ) je trajala 350 iteracij, druga faza ( $\pm 8^\circ$ ) 350 iteracij in zadnja faza ( $\pm 11^\circ$ ) 600 iteracij. Z večfaznim učenjem se robot nauči spoprijemanja z drugimi koti ob začetku delovanja strategije, kar izboljša njeno robustnost.

$$R = 1.0 \cdot p - 5.0 \cdot \theta^2 - 0.3 \cdot \Delta\theta^2 - 2 \cdot \Delta\varphi^2 - 0.4 \cdot v^2 - 0.15 \cdot x^2 - 0.1 \cdot j_{l+d} - 2.0 \cdot f j_{l+d} - 0.003 \cdot A_{M3} - 100 \cdot e \quad (1)$$

Trajanje epizode smo omejili z mejnimi vrednostmi treh spremenljivk. Epizoda se je zaključila, ko je bila absolutna vrednost naklona robota pet zaporednih korakov nad  $40^\circ$  ali nagib robota pod  $20^\circ$  ali nad  $72^\circ$ . Izpolnitev enega izmed teh pogojev je klasificirano kot padec. Če je robot uspešno ohranil naklon in nagib znotraj omejenih vrednosti, se je epizoda terminirala po 10 s. Tako vsaka epizoda pri frekvenci regulacije 50 Hz obsega maksimalno 500 regulacijskih korakov.

Tabela 1: Členi nagrajevanja agenta.

| Nagrada/Kazen                 | Utež                        |
|-------------------------------|-----------------------------|
| Preživetje                    | $+1.0 \cdot p$              |
| Naklon                        | $-5.0 \cdot \theta^2$       |
| Sprememba naklona             | $-0.3 \cdot \Delta\theta^2$ |
| Ukaz sklepov (L/D)            | $-0.1 \cdot j_{l+d}$        |
| Hitra sprememba sklepov (L/D) | $-2.0 \cdot f j_{l+d}$      |
| Tok koles                     | $-0.003 \cdot A_{M3}$       |
| Predčasna prekinitev          | $-100 \cdot e$              |
| Linearna hitrost              | $-0.4 \cdot v^2$            |
| Pozicija                      | $-0.15 \cdot x^2$           |
| Sprememba nagiba              | $-2 \cdot \Delta\varphi^2$  |

## 4 Priprava za prenos v realno okolje

Cilj tega prispevka ni simulacija sama po sebi ampak možnost prenosljivosti predlagane rešitve na realni sistem. Največji problem delovanja Sim2Real (angl. sim-to-real) je nepopolnost realnega okolja in strojne opreme, kar ni vedno mogoče doseči z virtualnim modelom. To rešimo z integracijo rekurzivne arhitekture GRU in strogim upoštevanjem omejitev strojne opreme. Upoštevali smo velikost pomnilnika mikrokontroler STM32F446RE, zaradi česar smo nevronske mreže omejili na le tri skrite sloje z 32 nevroni oz. GRU celicami (slika 3). Nevronska mreža s skritim slojem 32 GRU celic in dvema skritima slojema po 32 nevronov vsebuje približno 6600 parametrov, kar v 32-bitnem zapisu s plavajočo vejico znaša okoli 26 kB. To predstavlja zanemarljiv delež pomnilnika mikrokontroler STM32F446RE (512 kB bliskovnega pomnilnika in 128 kB SRAM), rezultati sklepanja pa zahtevajo manj kot 1 kB delovnega pomnilnika. En prehod skozi mrežo obsega več tisoč množenj in seštevanj, kar na jedru Cortex-M4 traja nekaj  $10 \mu s$ , kar je bistveno manj od regulacijskega koraka 20 ms. Za čim večjo računsko hitrost smo uporabili fiksno normalizacijo vhodov nevronske mreže s konstantami, za aktivacijsko funkcijo pa izbrali računsko nezahtevno funkcijo ReLU. Na realnem sistemu kontrolna zanka zaradi omejitev komunikacijskih vodil CAN in RS485 teče s frekvenco 50 Hz, zato smo regulacijo z nevronske mreže na enako frekvenco omejili tudi v simulaciji. Zaradi zaščite motorjev DDSM115 smo

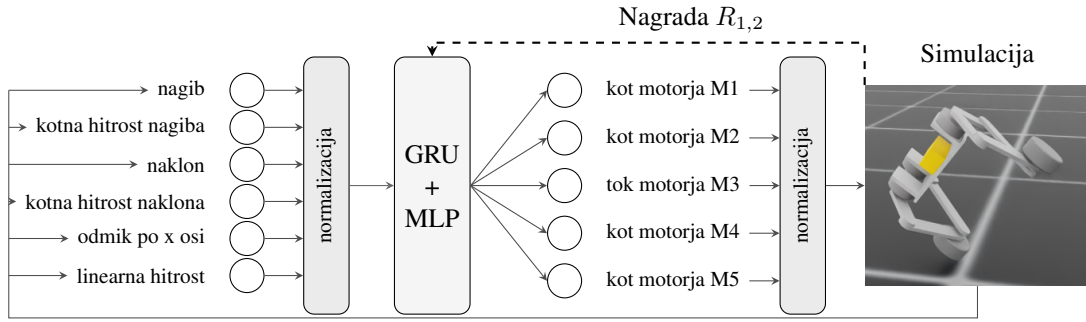
prav tako omejili izhod nevronske mreže za tok kolesa s funkcijo tanh, ki tok zvezno preslika v območje  $\pm 2,0$  A in tako prepreči preseganje varne tokovne meje.

Sim2Real smo želeli izboljšati še z dodajanjem randomizacije domene (angl. domain randomization), pri čemer celice GRU skozi sprotno posodabljanje skritega stanja delujejo kot implicitni identifikator spreminjajočega se sistema. Namesto da bi poskušali parametre realne platforme natančno izmeriti in poustvariti v simulaciji, strategijo učimo na celotni porazdelitvi verjetnih vrednosti, s čimer predvidevamo, da je velika možnost, da realna platforma deluje z vrednostmi, ki padejo znotraj te porazdelitve. Izdelali smo funkcijo, ki na začetku vsake epizode vpelje randomizacijo domene in naključno spreminja ojačenje motorjev (od 0,8 do 1,2), prag prenehanja delovanja motorjev zaradi trenja (od 0,03 do 0,20 A), odmik toka zaradi nesimetrije motorja (od  $-0,08$  do  $0,08$  A), časovno konstanto tokovne zanke (od 0,005 do 0,020 s), tokovno omejitev posameznega motorja (od 1,6 do 2,4 A), odmik meritve naklona zaradi napake v montaži ali kalibraciji IMU senzorja (od  $-0,5$  do  $0,5^\circ$ ) ter zakasnitev akcij na motorjih in zakasnitev opazovanj senzorjev (naključno 0 ali 1 korak). Prisotnost zakasnitev in nelinearnih шумov (Gaussov шум naklona s  $\sigma = 0,15^\circ$  in kotne hitrosti s  $\sigma = 0,02$  rad/s) v vsakem koraku simulacije dodatno utemeljuje uporabo arhitekture GRU, saj mreža s spominom uspešno kompenzira delno opazovanje stanja brez vnosa faznega zamika.

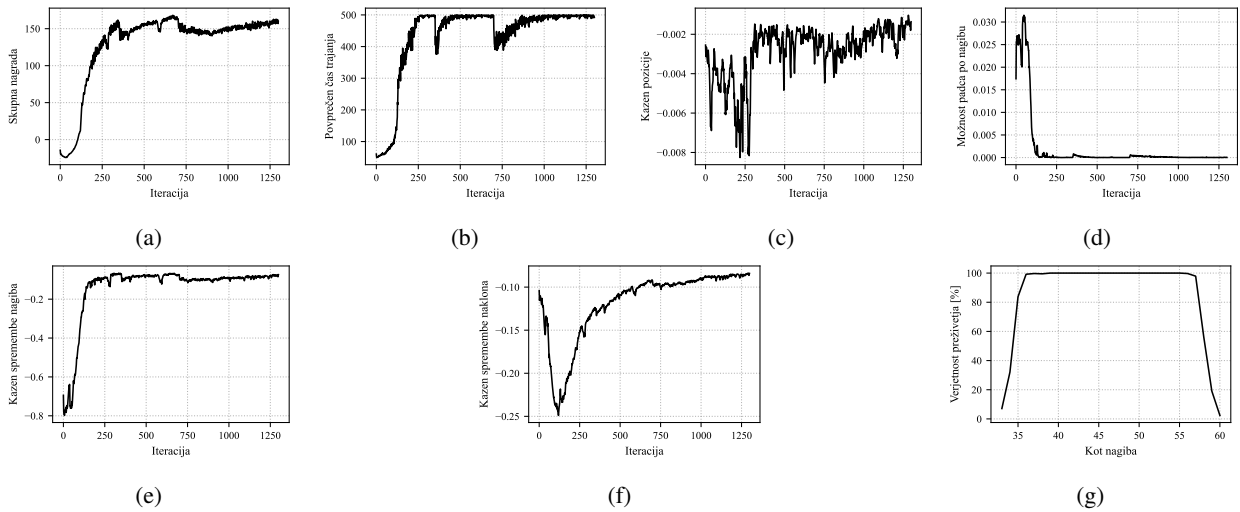
## 5 Eksperimentalni rezultati

Posluževanje večfaznega učenja je očitno vidno predvsem v grafih 4a, 4b in 4d kot kratkotrajni padci skupne nagrade in povprečnega trajanja epizode ter rahlo povečanje verjetnosti padca, čemur sledi počasnejša konvergenca nazaj proti maksimumu z vsako težjo fazo. Graf 4c teh prehodov ne izkazuje jasno, kar nakazuje stabilnost sistema znotraj posamezne faze. Ker robot ravnotežje ohranja s sočasnim delovanjem vseh petih motorjev (M1–M5), se neprestano rahlo pomika naprej, nazaj, gor in dol, zaradi česar kazni v grafih 4c, 4e in 4f konvergirajo proti majhni, a nikoli ničelni vrednosti — kar pomeni, da je robot ravnotežje našel in ga ohranjal znotraj majhnih odstopanj nagiba in naklona.

Za testiranje delovanja smo ustvarili simulacijsko okolje, v katerega smo robota postavili pod različnimi koti nagiba ter opazovali kolikšna je verjetnost, da najde in ohrani ravnotežno lego do konca epizode trajajoče 10 s. S tem smo želeli simulirati prehod iz dvokolesnega ravnovesja v enokolesno ravnovesno lego. Za večjo robustnost sistema smo primerjali verjetnost preživetja v velikem razponu začetnega nagiba ob začetku delovanja strategije. Testiranje je potekalo v 64 okoljih, kjer smo za vsak kot nagiba vsako okolje opazovali 6 epizod. Glede na graf 4g je strategija najuspešnejša med  $36^\circ$  in  $57^\circ$ , kjer dosega



Slika 3: Arhitektura strategije: 6-razsežni vektor stanja se najprej normira s fiksnimi konstantami, sledi obdelava z RNN nevronske mreže, normalizacija izhodnih veličin in simulacija. 3 izhodne krmilne veličine.



Slika 4: (a) Nagrada na epizodo, (b) Povprečen čas trajanja epizode, (c) Kazen pozicije na epizodo, (d) Možnost padca po nagiba v vsakem koraku, (e) Kazen spremembe kota nagiba, (f) Kazen spremembe kota naklona, (g) Verjetnost preživetja glede na začetni kot nagiba.

skoraj konstantno 100 % verjetnost preživetja. Tukaj se vidi povezava med učenimi vrednostmi in uspešnostjo delovanja, saj je bila strategija učena na maksimalnem razponu  $47^\circ \pm 11^\circ$ , kjer tudi deluje najbolj robustno. Takoj ko pa začetni naklon pade izven mej učenja pa se lahko opazi strm padec verjetnosti preživetja.

## 6 Zaključek

V prispevku smo obravnavali ravnotežje kolesno-nožnega robota na enem samem kolesu, ki predstavlja nestabilen, sklopljen in podaktuiran regulacijski problem z nagnjeno ravnovesno lego, ki ga s klasičnim modelom težko opišemo. Pokazali smo, da je takšen problem mogoče regulirati s strategijo učeno z globokim spodbujevanim učenjem, ki uporablja celice GRU kot osnovno arhitekturo. Strategija se je v mejnem območju učenja (začetni nagib  $47^\circ \pm 11^\circ$ ) izkazala zanesljivo. Ker je robustno območje delovanja tako veliko, je tudi prehod iz dvokolesnega v enokolesno ravnovesje lažje. Rešitev namenoma deluje v okviru omejitev ciljne strojne opreme z majhno mrežo, fiksno normalizacijo, frekvenco regulacije 50 Hz ter z randomizacijo domene, prilagojeno naključnosti realnega okolja, kar odpira pot neposrednemu prenosu na vgrajeni sistem v realnem okolju.

Nadaljevanje tega dela definitivno odpira vrata za uporabo takšnih robotskih sistemov v realnih nestrukturiranih območjih, kjer bi lahko s pomočjo podobnih inherentno nestabilnih gibov omogočilo mobilnim robotom boljše premagovanje ovir. Uporaba celic GRU bi tako lahko bila uporabljena za boljše napovedovanje naslednjih stanj in premagovanju nestabilnosti v okolju samem.

## Literatura

- [1] K. Wang *et al.*, "Wheeled-legged robots for multi-terrain locomotion in plateau environments," *Biomimetic Intelligence and Robotics*, vol. 5, no. 3, p. 100256, 2025.
- [2] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," *arXiv preprint arXiv:2109.11978*, 2022.
- [3] I. Smirnov and S. Gu, "RLBenchNet: The right network for the right reinforcement learning task," *arXiv preprint arXiv:2505.15040*, 2025.
- [4] J. Tofant and D. Gleich, "Diferenčno vodenje agilnega dvokolesnega mobilnega robota," in *Zbornik konference ERK 2025*, Portorož, Slovenija, 2025, pp. 597–600.
- [5] M. Mittal *et al.*, "Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning," *arXiv preprint arXiv:2511.04831*, 2025.